Mathematics of deep learning part 1
Report

Bruno PILARCZYK

Tanguy MAGON

M2 Statistics and Econometrics

Toulouse School of Economics

Contents

1 Dataset 1 **2**
1.1 Context . . . 2
1.2 Support Vector Machines . . . 2
1.3 Hard Margin Classification . . . 4
1.3.1 Primal form . . . 4
1.3.2 Dual form . . . 4
1.4 Soft Margin Classification . . . 5
1.5 Kernel . . . 5

2 Dataset 2 **6**
2.1 Context . . . 6
2.2 Data cleaning . . . 6
2.2.1 Missing values . . . 6
2.2.2 Transforming categorical data into dummies . . . 7
2.3 Descriptive statistics . . . 7
2.4 Data Preparation . . . 8
2.5 Regression models . . . 8
2.5.1 Ridge regression model . . . 8
2.5.2 LASSO regression model . . . 10
2.6 Model comparison . . . 11

1 Dataset 1

1.1 Context

The train dataset is composed of 5380 observations and 20 variables, 19 explanatory variables and 1 dependent variable ($n = 5380$, $p = 19$). y is a binary dependent variable which has to be predicted.

$$y_i = \begin{cases} 1 & \text{if } i \text{ presents a credit default} \\ 0 & \text{otherwise} \end{cases} \quad , \quad \forall i \in \{1, \dots, n\}$$

After training models, the aim will be to predict Y on the test dataset, containing 1345 observations. Here are the explanatory variables going to be used:

- Customer_Open_Date
- Prod_Closed_Date
- BirthDate
- Prod_Decision_Date
- Net_Annual_Income
- Number_Of_Dependant
- Years_At_Business
- Customer_Type
- P_Client
- Educational_Level
- Marital_Status
- Prod_Sub_Category
- Source
- Type_Of_Residence
- Prod_Category

1.2 Support Vector Machines

Instead of considering the set $\{0, 1\}$ as possible modalities for the y_i, we will consider the set of values $\{-1, 1\}$. The aim is to classify the observations based on the x_i by drawing an hyperplane. Here is the decision rule for the separable case:

$$y_i = \begin{cases} 1 & \text{if} \quad w^T x_i + b \geq 0 \\ -1 & \text{if} \quad w^T x_i + b < 0 \end{cases} \tag{1}$$

w represents the weight associated to the x_i and b the bias term (intercept), they will be both determined and computed later on.

It is possible to define two "routes" such that $w^T x + b = 1$ and $w^T x + b = -1$. We will denote the x_i such that $w^T x_i + b \geq 0$ by x_+ and the x_i such that $w^T x_i + b < 0$ by x_-.

Thus we obtain the following inequalities:

$$w^T x_+ + b \geq 1 \tag{2}$$

$$w^T x_- + b \leq -1 \tag{3}$$

It leads to:

$$y_i(w^T x_i + b) - 1 \geq 0 \quad , \quad \forall i \in \{1, \ldots, N\} \tag{4}$$

For x_i in a gutter:

$$y_i(w^T x_i + b) - 1 = 0 \tag{5}$$

Such observations satisfying the expression above are called the support vectors.
Now the aim is to determine the width of the route. It is well illustrated on this figure:

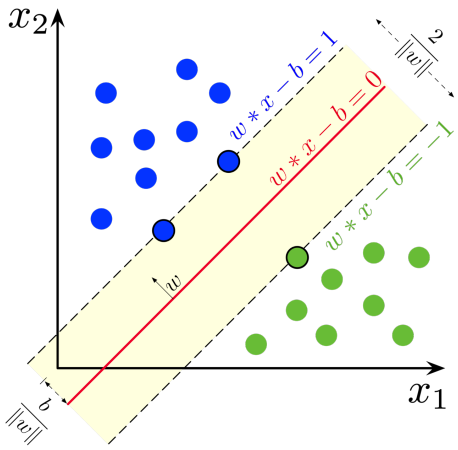

Figure 1: Support vector equations, street representation

Basically the width of the route would be just the difference between an x_+ and an x_-, both on the gutter from each side. Hence:

$$\text{width} = x_+ - x_- \tag{6}$$

In the above figure, x_+ are represented by the blue dots, the x_- by the green ones.

However it is necessary to normalize this width by multiplying the difference by the unit vector[1] of the weights: $\frac{w}{||w||}$. It gives:

$$\text{width} = (x_+ - x_-) \times \frac{w}{||w||} \tag{7}$$

From equation (5) we can obtain the following equalities by substituting Y_i by -1 or 1 whether the observation is characterized by x_- or x_+:

$$w^T x_+ + b = 1 \Leftrightarrow w^T x_+ = 1 - b \tag{8}$$

$$-w^T x_- - b = 1 \Leftrightarrow w^T x_- = -(1 + b) \tag{9}$$

Replacing (8) and (9) in (7) gives:

$$(1 - b + 1 + b) \times \frac{w}{||w||} = \frac{2}{||w||} \tag{10}$$

In order to separate well the instances, the goal is maximize the width and so $\frac{2}{||w||}$:

$$\max_w \frac{2}{||w||} \Leftrightarrow \min_w \frac{||w||}{2} \Leftrightarrow \min_w \frac{1}{2}||w||^2 \tag{11}$$

We prefer to use the squared norm because it is mathematically more convenient: the derivative is easier to calculate, indeed, $||w||$ is not differentiable at $w = 0$.
As for now, not any instance is allowed to be on the "wrong" side of the street, we are solving a **hard margin classification** problem.

[1] We commonly use the euclidian norm to deal with this problem, hence this latter will be used everytime we will use the norm of the weigths

1.3 Hard Margin Classification

1.3.1 Primal form

Here is the problem under constraint one should solve for this hard margin classifier:

$$\min_{w,b} \frac{1}{2}||w||^2 \quad \text{s.t.} : y_i(w^T x_i + b) - 1 \geq 0, \quad \forall i = 1, \ldots, n \tag{12}$$

It leads to minimize this lagrangian function in primal form, note that the α_i represents the lagrangian multipliers and there are as much as the number of instances to take into account, hence N.

$$\mathcal{L}(w, b, \alpha_1, \ldots, \alpha_n) = \frac{1}{2}||w||^2 - \sum_{i=1}^{n} \alpha_i (y_i(w^T x_i + b) - 1) \tag{13}$$

One can solve this problem (finding the minimum) by studying the Karash-Khun-Tucker conditions:

$$\begin{cases} \dfrac{\partial \mathcal{L}}{\partial w} = 0 \Leftrightarrow w = \sum_{i=1}^{N} \alpha_i y_i x_i \\ \\ \dfrac{\partial \mathcal{L}}{\partial b} = 0 \Leftrightarrow \sum_{i=1}^{N} \alpha_i y_i = 0 \end{cases}$$

The solution concerning w informs us that each w_k is a linear combination of the lagrangian multiplier and the product of the y_i & x_i.

1.3.2 Dual form

By substituting solutions found solving the lagragian in primal form, we can deduce the dual form lagrangian function which is more convenient to solve and which has to be maximized.

$$\begin{aligned} \mathcal{L}(b, \alpha_1, \ldots, \alpha_n) = & \frac{1}{2}(\sum_{i=1}^{n} \alpha_i y_i x_i)(\sum_{j=1}^{n} \alpha_j y_j x_j) \\ & - \sum_{i=1}^{n} \alpha_i y_i x_i \times (\sum_{j=1}^{n} \alpha_j y_j x_j) \\ & - b \sum_{i=1}^{n} \alpha_i y_i \\ & + \sum_{i=1}^{n} \alpha_i \end{aligned}$$

Note that thanks to the previous KKT conditions, the third term of the lagrangian equals vanishes, this gives: $-b \sum_{i=1}^{n} \alpha_i y_i = 0 \Rightarrow \sum_{i=1}^{n} \alpha_i y_i = 0$

This leads to the following lagrangian function to maximize:

$$\mathcal{L}(\alpha_1, \ldots, \alpha_n) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i x_j \tag{14}$$

After solving this problem we will get the α_i which permit us to compute $\hat{w} = \sum_{i=1}^{N} \alpha_i y_i x_i$. Afterwards b still needs to be calculated. To do so, we can use the fact that:

$$y_i(w^T x_i + b) - 1 = 0$$

In order to get $\hat{b}$, we can use a support vector (called a for the example):

$$\hat{b} = 1 - y_a(\hat{w}^T x_a)$$

We can have even more accurate results by taking the mean over all the k support vectors:

$$\hat{b} = \frac{1}{k}\sum_{i=1}^{k} 1 - y_i(\hat{w}^T x_i)$$

In order to allow instances to be on the wrong side of the street we have to allow some distances ζ_i. This is the method we used in our model and it is **soft margin classification** problem.

1.4 Soft Margin Classification

The only difference with hard margin classifiers is that we allow the individuals to cross the street of some distances $\zeta_i, \forall i = 1, \ldots, n$. We will adjust how much it has to be taken into account by adjusting an hyperparameter C, which is the cost of the constraints. This changes the constraints of the initial minimization problem:

$$\min_{w,b,\zeta_1,\ldots,\zeta_n} \frac{1}{2}||w||^2 + C\sum_{i=1}^{n}\zeta_i$$

$$\text{s.t.:} \begin{cases} y_i(w^T x_i + b) \geq 1 - \zeta_i \quad , \quad \forall i = 1, \ldots, n \\ \zeta_i \geq 0 \end{cases}$$

The first constraint can be written as:

$$\zeta_i \geq 1 - y_i(w^T x_i + b)$$

By taking into account the second constraint, we know that $\zeta_i \geq 0$. The combination of both constraint gives:

$$\max(1 - y_i(w^T x_i + b), 0) \quad , \quad \forall i = 1, \ldots, n$$

This constraint is also called as the **hinge loss**.
We can now write the lagrangian function to minimize in order to solve the problem:

$$\mathcal{L}(w, b, \alpha_1, \ldots, \alpha_n) = \frac{1}{2}||w||^2 - C\sum_{i=1}^{n}\alpha_i \max(1 - y_i(w^T x_i + b), 0) \tag{15}$$

Like before we will compute the first two KKT conditions in order to refer to a dual form of the this lagrangian and then obtain estimated values for w, b and the α_i.

1.5 Kernel

Support vector machines are not only limited to compute a straight line which classifies observations, for our project we used kernels. Indeed instead of taking x into account we can take a function $\Phi(.)$ called a kernel. For our project we tested the linear kernel as well as the Radial Basis Function kernel.

$$\Phi(x, x') = x^T x' \tag{16}$$

Figure 2: Linear kernel

$$\Phi(x, x') = \exp^{\gamma||x-x'||^2} \tag{17}$$

Figure 3: Radial Basis Function kernel

2 Dataset 2

2.1 Context

The original dataset is composed of 1460 observations and 81 variables (ID variables excluded). It contains both qualitative and quantitative variable types, which have to be treated and cleaned separately. First we begin splitting the dataset in both train and test set by creating a dummy variable which indicates if the observation belongs to the test set or not. The train set is composed of 1168 observations and the test set contains 292 instances.

2.2 Data cleaning

We computed 6965 missing values within the dataset.

2.2.1 Missing values

We delete variables which contains too much missing values: Alley, FireplaceQu, PoolQC, Fence, MiscFeature

For the quantitative variables, as they do not contain so much missing values, we executed a median imputation. It concerned: LotFrontage, GarageYrBlt, MasVnrArea

The median of a variable x is given by:

$$q_{0.5}(x) = \inf\{t \in \mathbb{R} \text{ such that } \sum_{i=1}^{n} w_i \mathbb{1}_{(-\infty,t]}(x_i) \geq 0.5\}$$

For the qualitative variables containing missing values, first we turn them into factor before doing the k-nearest neighbors method in order to impute the missing values. We chose $k = 5$ neighbors because it is an odd number and it helps finding the mode the most represented among the nearest neighbors.

Algorithm 1: K-nearest neighbors

Input: dataset: *houses*, distance function d, $k = 5 \in \mathbb{N}$

Output: Inputation of all missing values

for $i \in \{1, \ldots, n\}$ **do**
  Calculate all the euclidian distances d_e between i and any other $j \neq i \in \{1, \ldots, n\}$:
  $d_e(x_i, x_j) = \sqrt{\sum_{m=1}^{p} (x_{i,m} - x_{j,m})^2} = ||x - y||_2 \quad ; \quad x_i, x_j \in \mathbb{N}^p$
  Keep the 5 nearest neighbors of *houses* according to the distance function d_e
  As we are dealing with categorical data we will compute the mode of the 5 nearest observations kept according to the distance function d_e
  Impute the missing value by the predicted mode calculated
end for

We can add that according to the algorithm in **R**, the data is scaled, any observation has its variables divided by their standard deviation:

$$x_i^{'} = \frac{x_i}{\sigma_x}$$

Even if we deal with categorical data, euclidian distance works well as we transformed these variables into factors: $x_l \in \{0, 1\}$ for dummies, $x_l \in \mathbb{N}$ for qualitative variables with strictly more than 2 modalities.

2.2.2 Transforming categorical data into dummies

As categorical data in this dataset have modalities in common between variables (*e.g.* Exterior1st and Exterior2nd have most of their modalities in common), we cannot used packages for one-hot encoding or FastDummies.

"houses" is the dataset, "columns_cat" is the list of all categorical variables. So here is the double-loop algorithm we did to do so:

Algorithm 2: Customized one-hot encoder

Input: *columns_cat*: list of categorical variables, *houses*: dataset.

Output: Dummies for each modality (minus one) of each categorical variable

for *variable* $\in$ *columns_cat* **do**
  extract the list of modalities of *variable*: *list_modalities*

  for *word* $\in$ *list_modalities* **do**
    $$\text{"}word_variable\text{"} = \begin{cases} 1 \text{ if } word = variable \\ 0 \text{ otherwise} \end{cases}$$
  end for
  Delete the last dummy variable created in *houses*
  Delete *variable*
end for

One can notice that we delete the last created dummy, it is done in order to prevent further regressions from collinearity issues. Afterwards, we delete the original categorical variable before encoding the next variable. In order to solve the original problem (giving the same name to dummies variable because of same modality names), we create a dummy variable according to its modality name ("*word*" in the algorithm) and the categorical variable it comes from ("*variable*" in the algorithm).

Finally we end up with 233 variables: 232 explanatory variables + 1 dependent variable.

2.3 Descriptive statistics

Here is the distribution of the dependent variable represented by an histogram of SalePrice:

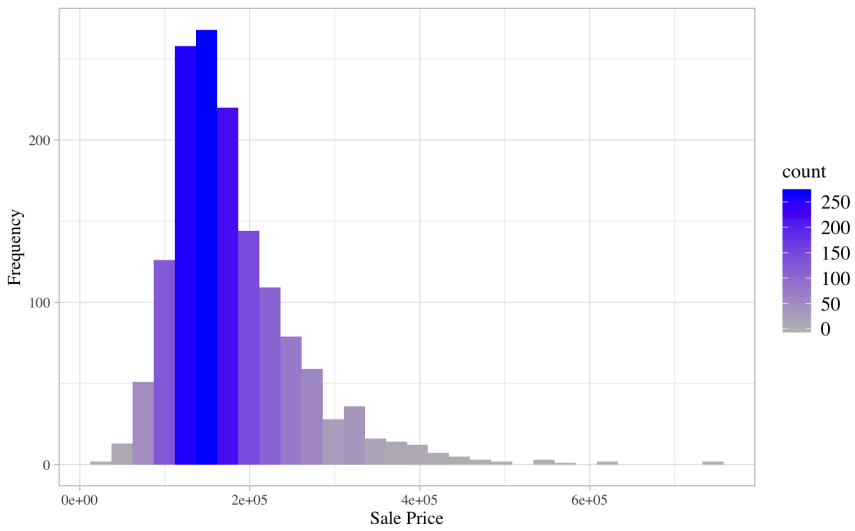

Figure 4: Distribution of the dependent variable SalePrice

Other statistics about the 232 explanatory variables are described in the Rmarkdown using the `summary()` function.

2.4 Data Preparation

In order to avoid overfitting we can use the k-fold method, it splits the train set k distinct subset of the original data set, this resampling method is insuring random repartition between all the folds. We chose 4 folds.
In other words, at each iteration, the following models we are going to test will train on one fold and test it against the other ones. As we chose 4-folds, it will do it four times, so that the entire dataset has been passed through the algorithm.

2.5 Regression models

2.5.1 Ridge regression model

As there are 232 explanatory variables, it is likely that some variables are not useful. The Ridge model acts like a selection variable model. From now on, n will denote the size of the train set, not the entire dataset.
Here is the loss function the Ridge model is supposed to minimize:

$$\sum_{i=1}^{n}(y_i - a - < x_i, \beta >)^2 + \lambda||\beta||_2^2$$

Notice we include an intercept term denoted by a. The last term of the equation is the penalization term: it is the regularization parameter λ times the squared norm of the coefficients.

In order to increase the effiency of the training, data has been standardized (centered and scaled):

$$\frac{x_{i,j} - \overline{x_j}}{\sigma_{x_j}} \quad , \quad \forall i \in \{1, \ldots, n\}, \forall j \in \{1, \ldots, p\}$$

In matrix form, the Ridge regression solves:

$$\hat{\beta}_{\text{Ridge}} := arg \min_{a \in \mathbb{R}, \beta \in \mathbb{R}^p} ||y - a - x^T\beta||_2^2 + \lambda||\beta||_2^2$$

By a cross-validation procedure we are able to find the optimal value for lambda that minimizes the Root Mean Squared Error:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y_i})^2}$$

We found $\lambda_{\text{opt}} = 39442.06$

Here is the plot showing the RMSE depending on the value taken by the regularization term λ:

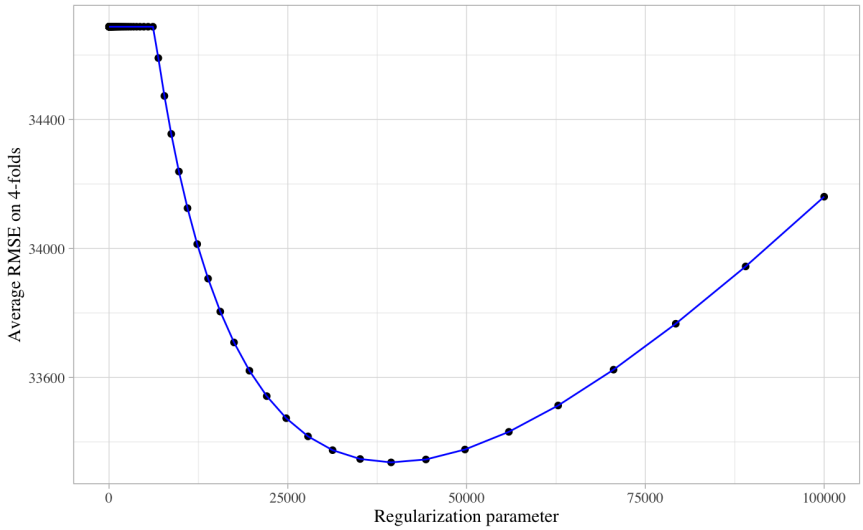

Figure 5: Average RMSE on the 4-folds depending on λ (Ridge regression model)

It was also relevant to plot the residuals depending on the fitted values:

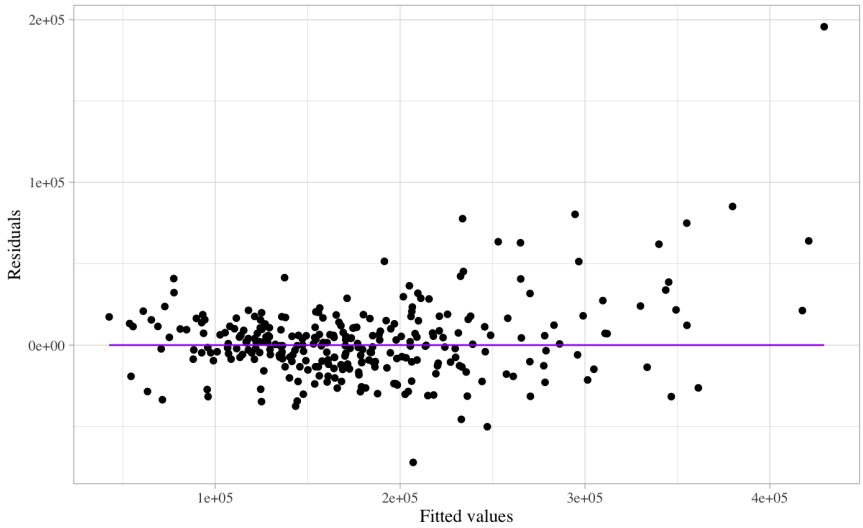

Figure 6: Residuals depending on the fitted values (Ridge regression model)

We found those values once the Ridge regression model has been applied on both the entire train set and the test set.

$$\begin{cases} \text{RMSE}_{\text{test}} = \sqrt{\dfrac{1}{n} \displaystyle\sum_{i \in \text{train}} (y_i - \hat{y}_i)^2} = 25267.03 \\ \text{RMSE}_{\text{test}} = \sqrt{\dfrac{1}{n} \displaystyle\sum_{i \in \text{test}} (y_i - \hat{y}_i)^2} = 24014.92 \end{cases}$$

2.5.2 LASSO regression model

Oppositely to the Ridge regression model, the *Least Absolute Shrinkage and Selection Operator* makes the coefficients be exactly equal to 0 if the value taken by the regularization parameter exceeds a certain threshold. It is due to the penalization term whose only difference is that it uses the ℓ_1 norm (instead of the ℓ_2 norm present in the Ridge regression model).

In order to increase the effiency of the training, data has been standardized (centered and scaled):

$$\frac{x_{i,j} - \overline{x_j}}{\sigma_{x_j}} \quad , \quad \forall i \in \{1, \dots, n\}, \forall j \in \{1, \dots, p\}$$

The LASSO regression model solves:

$$\hat{\beta}_{\text{LASSO}} := arg \min_{a \in \mathbb{R}, \beta \in \mathbb{R}^p} ||y - a - x^T \beta||_2^2 + \lambda ||\beta||_1$$

Due to convexity issues, this minimization problem can be solved with a Minimize Majorization algorithm.

By a cross-validation procedure we are able to find the optimal value for lambda that minimizes the Root Mean Squared Error. We found $\lambda_{\text{opt}} = 1873.817$

Here is the plot showing the RMSE depending on the value taken by the regularization term λ:

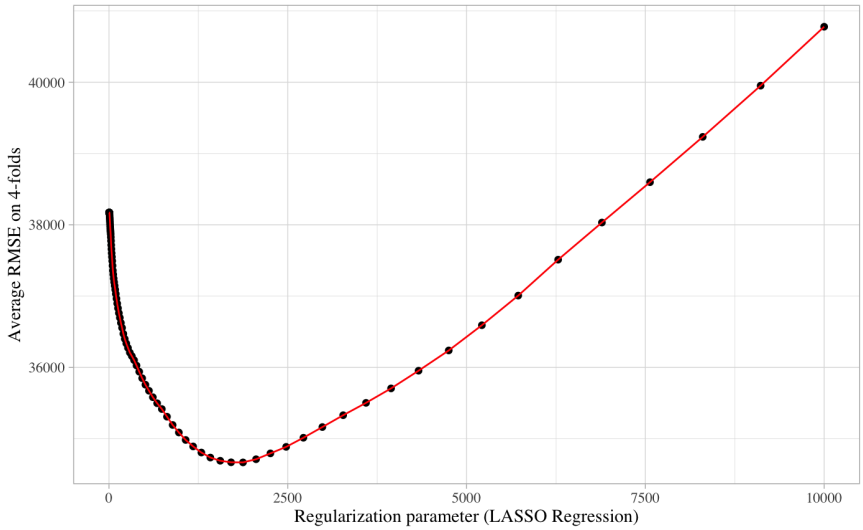

Figure 7: Average RMSE on the 4-folds depending on λ (LASSO regression model)

It was also relevant to plot the residuals depending on the fitted values:

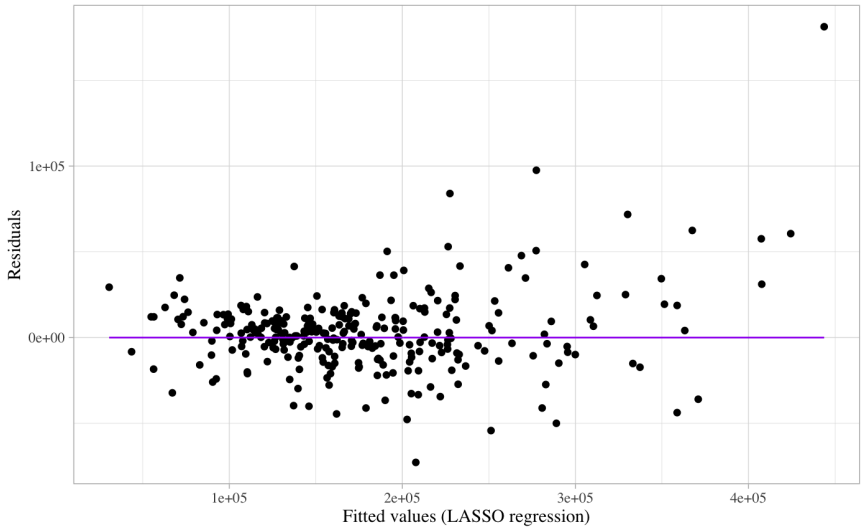

Figure 8: Residuals depending on the fitted values (LASSO regression model)

We found those results once the Ridge regression model has been applied on both the entire train set and the test set.

$$\begin{cases} \text{RMSE}_{\text{test}} = \sqrt{\dfrac{1}{n} \displaystyle\sum_{i \in \text{train}} (y_i - \hat{y}_i)^2} = 25853.34 \\ \text{RMSE}_{\text{test}} = \sqrt{\dfrac{1}{n} \displaystyle\sum_{i \in \text{test}} (y_i - \hat{y}_i)^2} = 23581.67 \end{cases}$$

2.6 Model comparison

Even if the Ridge regression model works better on the 4-folds, the LASSO regression model shows improved results on the test set. The difference in terms of RMSE is about -433.25.